

CSEN604: Data Bases II

Mohamed Karam Gabr

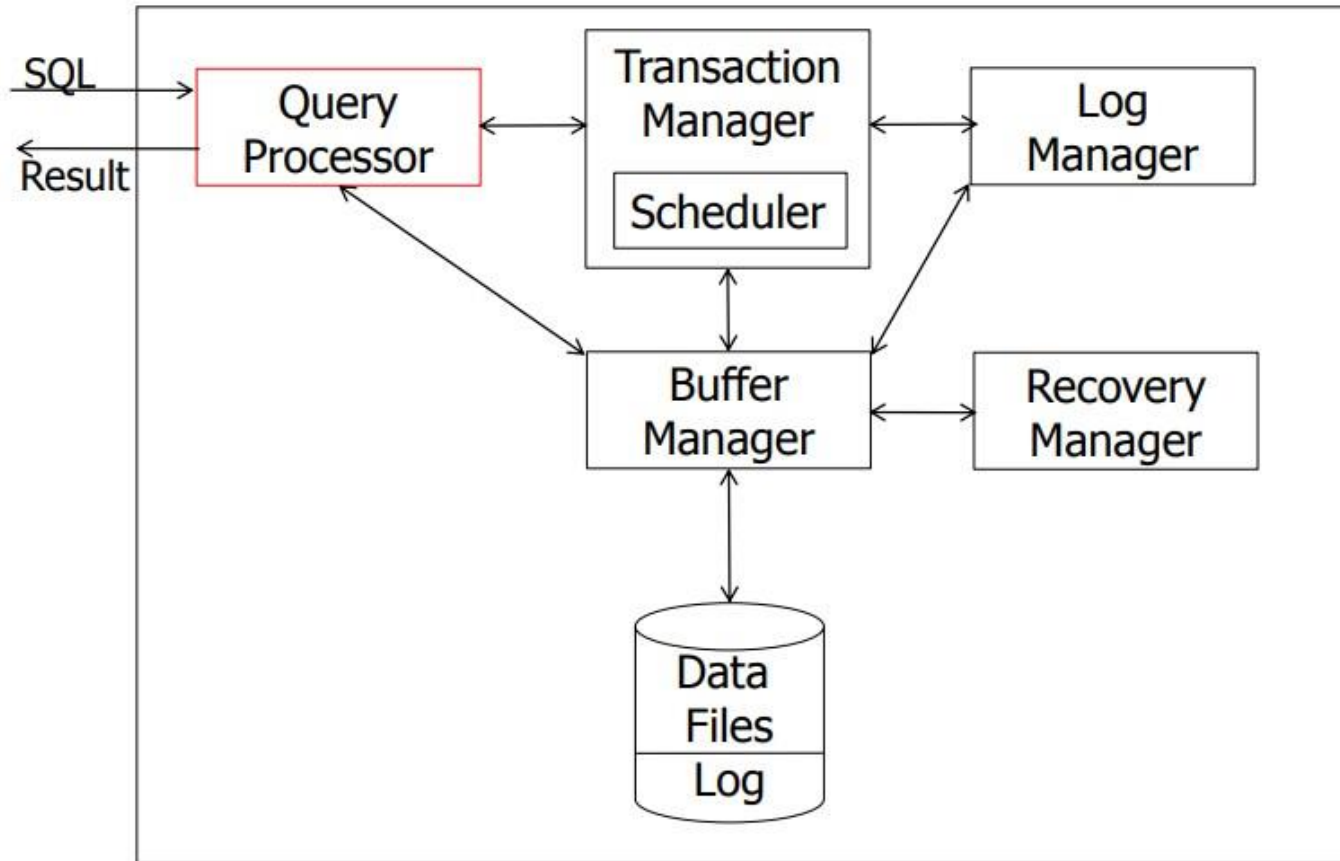
Query Optimization

Spring 2025

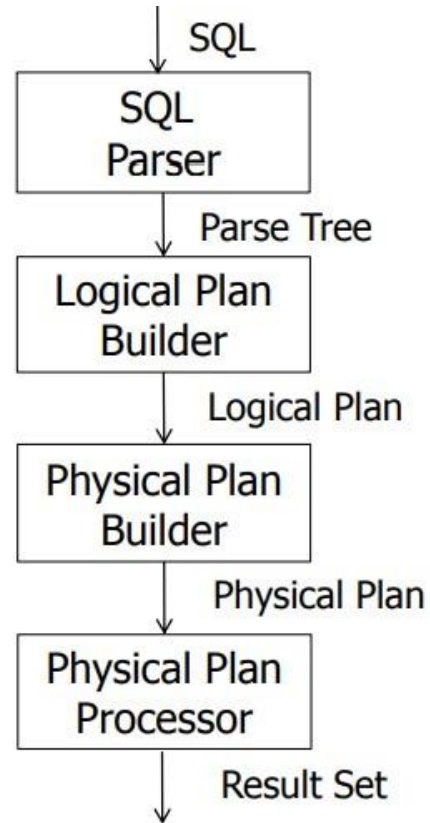
Outline

- Introduction.
- Relational Algebra/Logical Plan Optimization.
- Heuristic Optimization.

Introduction



Introduction



Introduction

StarsIn

title	starName	...	

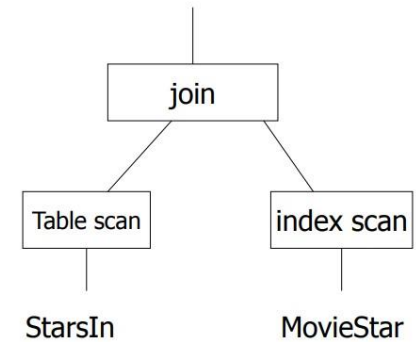
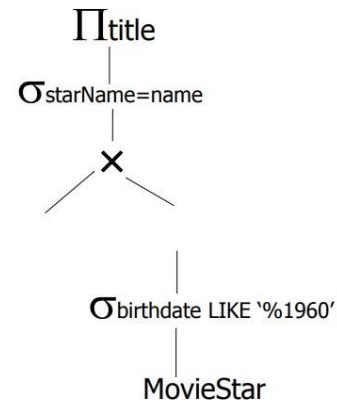
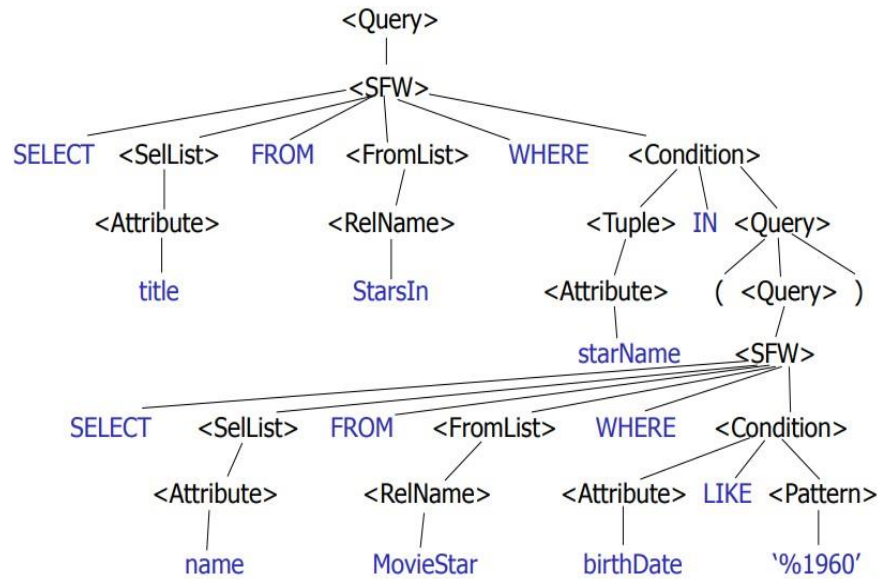
MovieStar

name	birthdate	...	

(Find the movies with stars born in 1960)

```
SELECT title
FROM StarsIn
WHERE starName IN (
    SELECT name
    FROM MovieStar
    WHERE birthdate LIKE '%1960'
);
```

Introduction



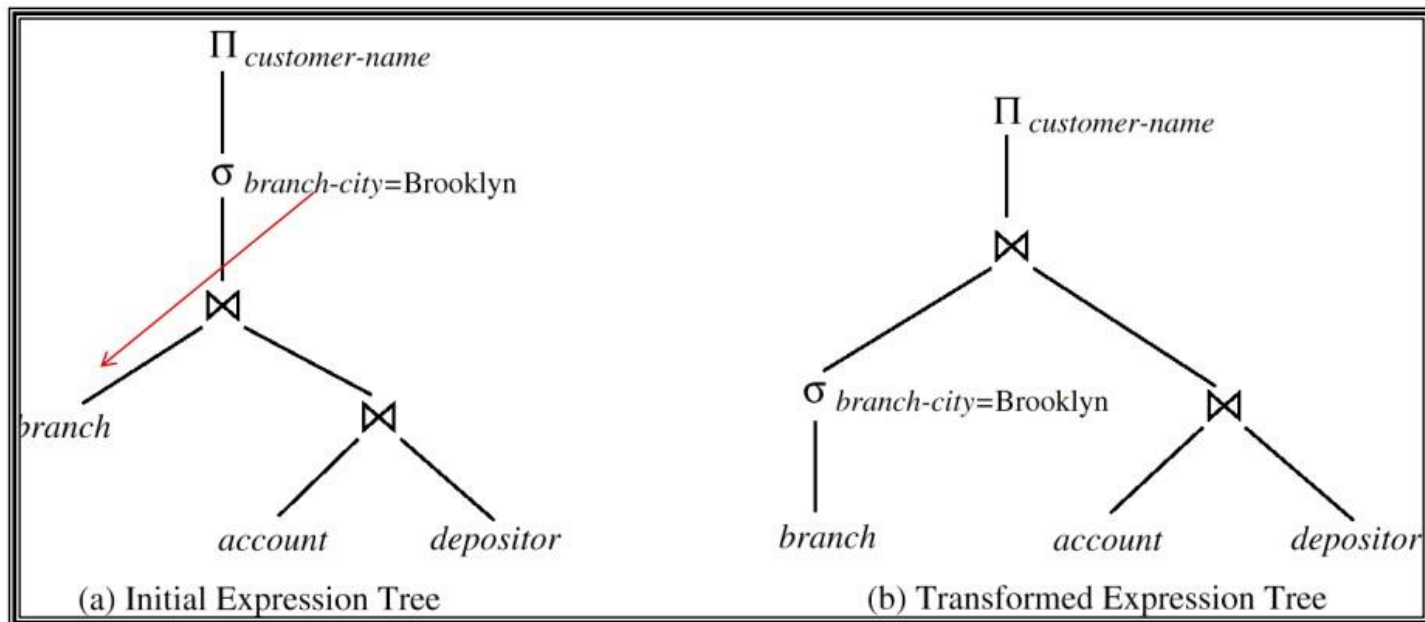
Relational Algebra/Logical Plan Optimization

- Relational Algebra is procedural language and gives you a certain order for executing a query.
- By applying certain rules, you can rearrange the order of executing the Query without changing the result.
- Such reordering can incredibly improve efficiency.
- For even a simple SQL, there are several alternative relational algebra trees – all produce the same result set. One of them will be optimal.
- There are 2 types of optimizers:
 - Heuristic-based Optimizers:
 - Apply greedy rules that aims to improve plan performance.
 - Limited, not always yield optimal plan, not popular anymore, but some RDBMS still use it.
 - Cost-based Optimizers:
 - Use a cost model to estimate cost of each plan and select cheapest cost.

Heuristic Optimization

- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - **Perform selection early (reduces the number of tuples).**
 - Perform projection early (reduces the number of attributes).
 - Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.

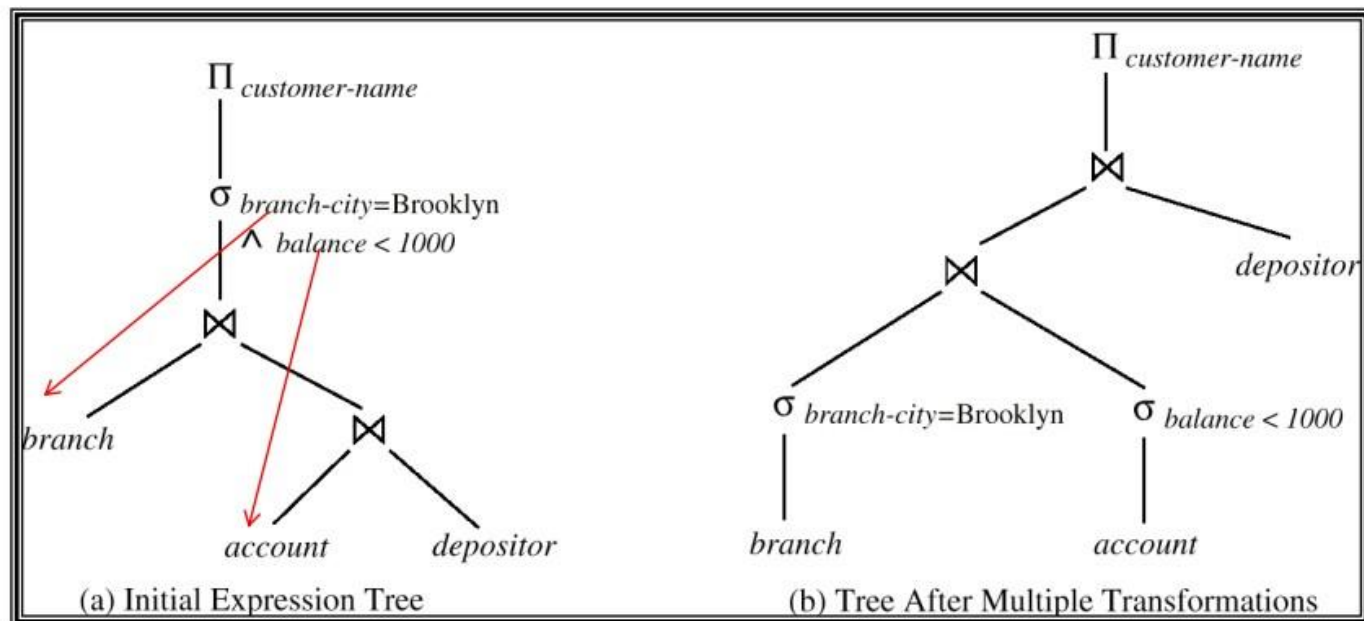
SELECT account.customer-name **FROM** branch,account,depositor **WHERE** branch.branch-city = "Brooklyn" **AND** Branch.id = account.branch_id **AND** account.id = depositor.account_id;



Heuristic Optimization

- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - **Perform selection early (reduces the number of tuples).**
 - Perform projection early (reduces the number of attributes).
 - Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.

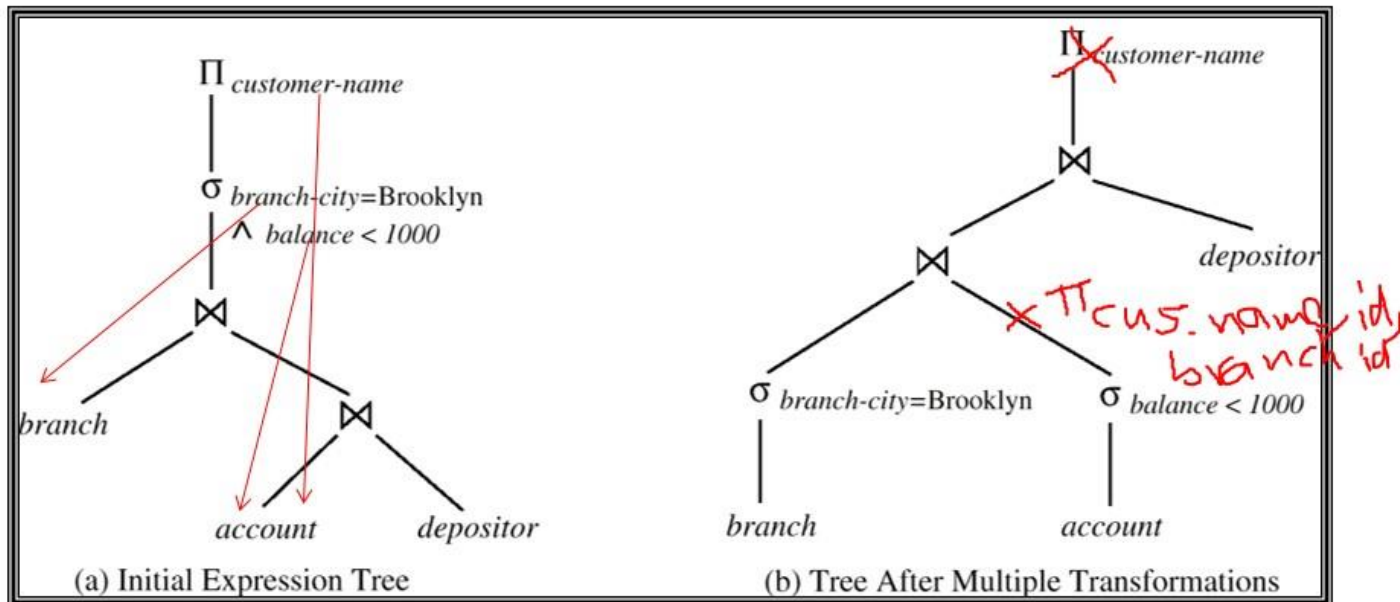
SELECT account.customer-name **FROM** branch,account,depositor **WHERE** branch.branch-city = "Brooklyn" **AND** Branch.id = account.branch_id **AND** account.id = depositor.account_id **AND** account.balance < 1000;



Heuristic Optimization

- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - Perform selection early (reduces the number of tuples).
 - **Perform projection early (reduces the number of attributes).**
 - Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.

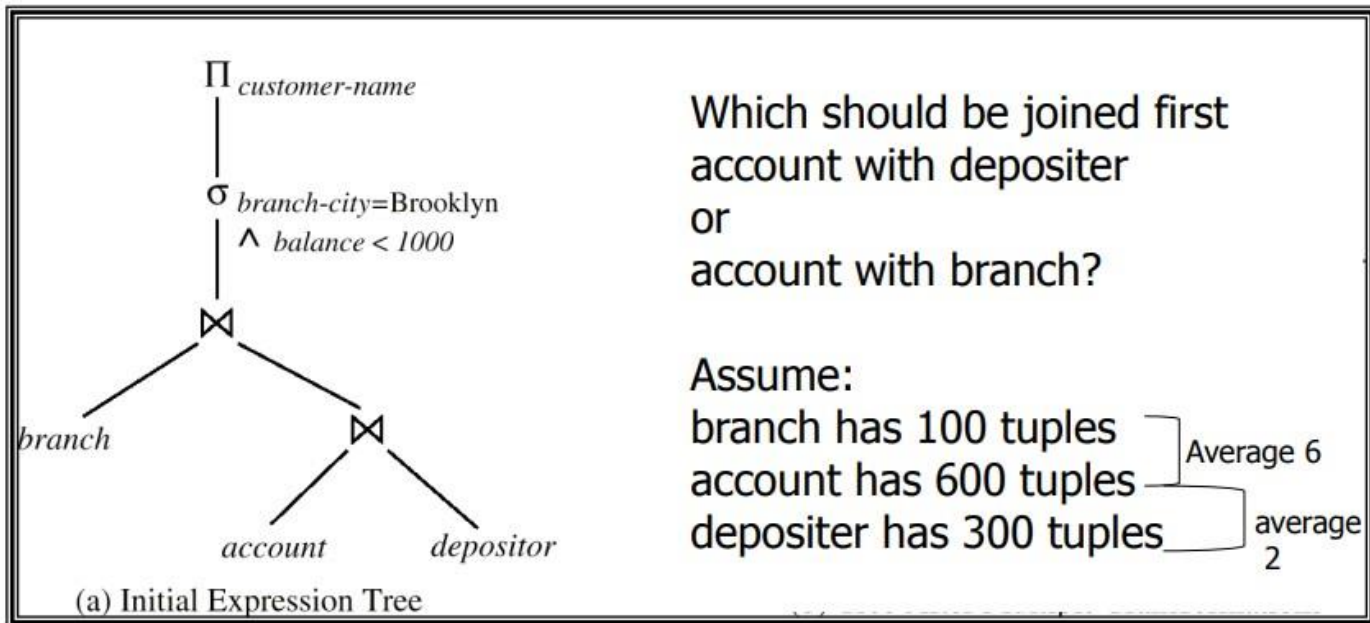
SELECT account.customer-name **FROM** branch,account,depositor **WHERE**
branch.branch-city = "Brooklyn" **AND** Branch.id = account.branch_id **AND**
account.id = depositor.account_id and account.balance < 1000;



Heuristic Optimization

- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - Perform selection early (reduces the number of tuples).
 - Perform projection early (reduces the number of attributes).
 - **Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.**

SELECT account.customer-name **FROM** branch,account,depositor **WHERE** branch.branch-city = "Brooklyn" **AND** Branch.id = account.branch_id **AND** account.id = depositor.account_id and account.balance < 1000;



Heuristic Optimization

- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - Perform selection early (reduces the number of tuples).
 - Perform projection early (reduces the number of attributes).
 - **Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.**

SELECT account.customer-name **FROM** branch,account,depositor **WHERE**
branch.branch-city = "Brooklyn" **AND** Branch.id = account.branch_id **AND**
account.id = depositor.account_id and account.balance < 1000;

